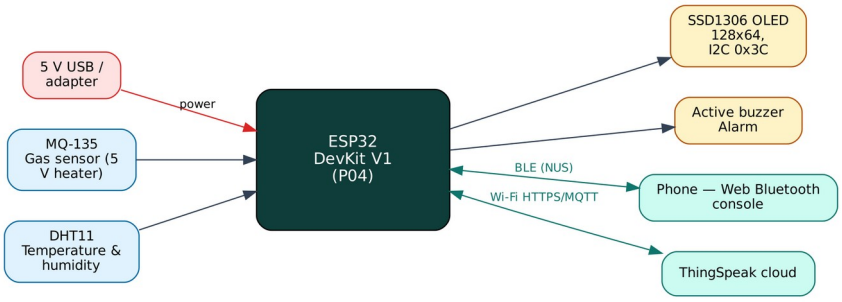
A PROJECT REPORT ON

Indoor Air Quality Monitor with Alarm

IoT project using ESP32 with Bluetooth Low Energy and Wi-Fi (ThingSpeak)



Project: P04 · Intermediate level

Peripherals: MQ-135 gas sensor, DHT11, 0.96" SSD1306 OLED, Active buzzer

Submitted by: _____

Under the guidance of: _____

Department / Institution: _____

October 2026

Certificate

This is to certify that the project report entitled **"Indoor Air Quality Monitor with Alarm"** is a bonafide record of the work carried out by _____ (Roll No. _____) in partial fulfilment of the requirements for the award of _____ in _____ during the academic year _____.

Project Guide: _____ Head of Department: _____

Date: _____ Place: _____

Acknowledgement

I express my sincere gratitude to my project guide _____ for continuous guidance and support, to the Head of the Department _____, and to all faculty members and friends who helped me complete this project successfully.

Contents

Abstract 6

1. Introduction 7

1.1 Problem Statement 7
1.2 Objectives 7
1.3 Scope of the Project 7
1.4 Organisation of the Report 7

2. Existing System and Proposed System 8

2.1 Existing Systems 8
2.2 Proposed System 8
2.3 Why Both BLE and Wi-Fi? 8

3. System Design 9

3.1 Block Diagram 9
3.2 System Architecture 9
3.3 Hardware Components and Cost 9
3.4 ESP32 DevKit V1 10
3.5 Working Principle of Components 10

4. Circuit Design 11

4.1 Circuit Diagram 11
4.2 Pin Connections 11
4.3 Interfacing and Power Notes 12

5. Data Flow Diagrams 13

5.1 DFD Level 0 (Context Diagram) 13
5.2 DFD Level 1 14
5.3 Data Dictionary 15

6. Software Design and Implementation 16

6.1 Development Environment 16
6.2 Firmware Architecture 16
6.3 Working of the System 16
6.4 Flowchart 17
6.5 BLE Communication Protocol 18
6.6 Cloud Integration (ThingSpeak) 18
6.7 Configurable Settings 19

7. Setup and Deployment Guide 20

7.1 Hardware Assembly 20
7.2 Flashing the Firmware 20
7.3 Cloud Setup (ThingSpeak) 21
7.4 BLE App Setup (Web Bluetooth Console) 21
7.5 First-Run Checklist 22

8. Testing and Results 23

8.1 Expected Output 23
8.2 Troubleshooting 23

9. Discussion 24

9.1 Advantages 24
9.2 Limitations 24
9.3 Safety Precautions 24
9.4 Security Considerations 24
9.5 Future Scope 24

10. Conclusion..... 25
References..... 26
Appendix A: Project Source Code..... 27
Appendix B: Shared Library Code..... 31

Abstract

Indoor air in kitchens, classrooms, offices and bedrooms can contain high levels of carbon dioxide, smoke and volatile organic compounds. This project uses an MQ-135 metal-oxide gas sensor to estimate a CO_2-equivalent concentration, compensates the sensor reading for temperature and humidity measured by a DHT11, and classifies air quality into five levels from Good to Hazardous. Values are shown on an OLED display, streamed to a phone over Bluetooth Low Energy and logged to ThingSpeak for long-term trends. A buzzer warns when the concentration exceeds an adjustable limit and can be muted from the phone. A one-tap calibration in clean outdoor air stores the sensor's reference resistance in flash.

Keywords: ESP32, Internet of Things, Bluetooth Low Energy, Wi-Fi, ThingSpeak, MQ-135 gas sensor, DHT11, 0.96" SSD1306 OLED, Active buzzer.

1. Introduction

The Internet of Things (IoT) connects physical devices — sensors and actuators — to networks so that data can be collected, analysed and acted on automatically. The ESP32 from Espressif Systems has made IoT affordable: a single ₹350 board combines a dual-core 240 MHz processor, Wi-Fi and Bluetooth Low Energy. This project uses both radios at once: BLE for direct, local interaction with a phone and Wi-Fi for cloud connectivity through ThingSpeak, a free IoT/cloud platform.

1.1 Problem Statement

Poor ventilation raises CO_2 and pollutant levels, causing headaches, drowsiness and reduced concentration; gas leaks and smoke can be dangerous. Dedicated NDIR CO_2 monitors cost ₹4,000 or more, so most homes and schools have no way to know when to open windows or switch on exhaust fans.

1.2 Objectives

1. Estimate indoor air quality as a CO_2-equivalent value with a low-cost MQ-135 sensor.
2. Compensate for temperature and humidity and smooth the readings.
3. Classify air quality into five levels and alarm above a set threshold.
4. Display values locally and on a phone, and log them to the cloud.
5. Allow field calibration and threshold changes without reprogramming.

1.3 Scope of the Project

Indoor rooms of up to about 30 m^2. The MQ-135 gives an indicative value, not a certified measurement; it is not a substitute for a gas-leak detector certified for LPG safety.

1.4 Organisation of the Report

Chapter 2 reviews existing approaches and describes the proposed system. Chapter 3 covers system design and components, Chapter 4 the circuit, Chapter 5 the data flow diagrams, Chapter 6 the software, Chapter 7 the step-by-step setup of hardware, cloud and app, Chapter 8 testing, Chapter 9 discussion and Chapter 10 the conclusion. Complete source code is given in the appendices.

2. Existing System and Proposed System

2.1 Existing Systems

Most MQ-135 projects print the raw ADC value or use the library's default reference resistance, which can be off by a factor of two or more. They ignore humidity, which strongly affects metal-oxide sensors, and do not handle the sensor's warm-up period.

2.2 Proposed System

The proposed system improves on these in the following ways:

- Datasheet-based temperature/humidity correction of the sensor resistance.
- Calibration against today's outdoor CO_2 (~420 ppm) with R0 saved in flash.
- Warm-up handling: no classification or uploads for the first 2 minutes.
- Five-level classification with phone notification and mutable buzzer alarm.

2.3 Why Both BLE and Wi-Fi?

Wi-Fi gives the device internet access for remote monitoring, history and control from anywhere, but it depends on a router and an internet connection, and the device needs the Wi-Fi password before it can connect. BLE needs no infrastructure: any modern phone can connect directly within about 10-30 m. Using both gives:

- **Provisioning** — Wi-Fi credentials and cloud keys are sent from the phone over BLE and stored in flash; nothing secret is hard-coded or uploaded to GitHub.
- **Resilience** — local monitoring and control keep working when the internet or router is down.
- **Remote access** — the cloud dashboard works from anywhere when the internet is available.
- **Low cost** — no extra hardware; both radios are built into the ESP32.

The ESP32 shares one 2.4 GHz radio between Wi-Fi and Bluetooth by time-division coexistence. For this to work the Wi-Fi modem-sleep mode must stay enabled, which the firmware ensures. NimBLE, a lightweight BLE stack, is used instead of the default Bluedroid stack to leave more RAM for TLS (HTTPS/MQTTS) connections.

3. System Design

3.1 Block Diagram

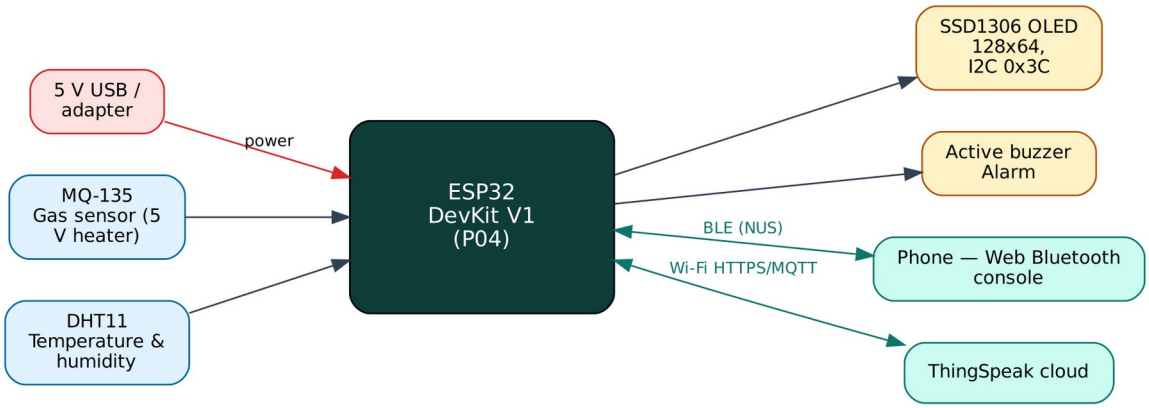


Figure 1: Block diagram of the Air Quality Monitor

The ESP32 is the central controller. Input devices (blue) feed measurements to it, output devices (amber) are driven by it, and two communication links (green) connect it to the user's phone over BLE and to the ThingSpeak cloud over Wi-Fi. The board is powered from 5 V through its USB connector or VIN pin.

3.2 System Architecture

The system follows the standard three-layer IoT architecture:

- **Perception layer** — MQ-135 gas sensor, DHT11, 0.96" SSD1306 OLED, Active buzzer connected to the ESP32 GPIO, ADC, I2C, SPI or UART.
- **Network layer** — BLE 4.2 GATT server (Nordic UART Service) and Wi-Fi 802.11 b/g/n with TLS-encrypted HTTPS REST to ThingSpeak.
- **Application layer** — the Web Bluetooth console on the phone and the ThingSpeak dashboard / database.

3.3 Hardware Components and Cost

#	Component	Qty	Specification	Cost
1	ESP32 DevKit V1 (ESP-WROOM-32, 30-pin)	1	Dual-core 240 MHz, Wi-Fi + BLE, 4 MB flash	₹350
2	Breadboard 830-point + jumper wires (M-M, M-F)	1	Prototyping and connections	₹150
3	Micro-USB data cable	1	Power + programming	₹60
4	MQ-135 air quality sensor module	1	NH_3, NOx, alcohol, benzene, smoke, CO_2; 5 V heater	₹120
5	DHT11 temperature & humidity module	1	For compensation	₹60
6	0.96" SSD1306 OLED 128x64 (I2C)	1	Address 0x3C	₹170
7	Active buzzer 5 V	1	Alarm	₹15
8	Resistors 10 kΩ + 20 kΩ	1	Divider for MQ-135 AO	₹5

#	Component	Qty	Specification	Cost
	Total (approx.)			**₹930**

Table 1: Bill of materials with approximate Indian prices (Robu.in / Amazon.in / local market, 2026)

> Prices are indicative and vary by seller. The breadboard, cable and ESP32 can be reused across all ten projects, so the incremental cost of this project is about ₹370.

3.4 ESP32 DevKit V1

Feature	Value
Module	ESP-WROOM-32 (ESP32-D0WD), dual-core Xtensa LX6 up to 240 MHz
Memory	520 KB SRAM, 4 MB SPI flash
Wireless	Wi-Fi 802.11 b/g/n 2.4 GHz; Bluetooth 4.2 BR/EDR + BLE
I/O	25 usable GPIO on the 30-pin board, 12-bit ADC (ADC1: GPIO32-39), 2 × DAC, PWM on any output
Interfaces	3 × UART, 2 × I2C, 3 × SPI, I2S, touch, RMT
Supply	5 V via USB/VIN, on-board 3.3 V regulator (AMS1117)
Logic level	3.3 V — inputs are NOT 5 V tolerant

Table 2: Key specifications of the ESP32 DevKit V1

3.5 Working Principle of Components

MQ-135

A tin dioxide (SnO_2) layer heated to about 200-300 °C has low conductivity in clean air. Reducing gases react with adsorbed oxygen on the surface and lower its resistance Rs. The module places the sensor in series with a load resistor RL, so Vout = Vc·RL/(Rs+RL). The datasheet's log-log curve gives ppm = a·(Rs/R0)^−b, with a = 116.6 and b = 2.769 for CO_2.

Compensation

Rs falls as temperature and humidity rise. The datasheet curves are approximated by a quadratic (below 20 °C) or linear (above 20 °C) correction factor, and Rs is divided by this factor.

SSD1306 OLED

A 128x64 monochrome OLED with an on-chip controller and frame buffer, driven over I2C. Each pixel emits light, so no backlight is needed and contrast is high.

DHT11 & buzzer

The DHT11 supplies temperature and humidity for compensation; the active buzzer sounds short beeps while the alarm is active.

4. Circuit Design

4.1 Circuit Diagram

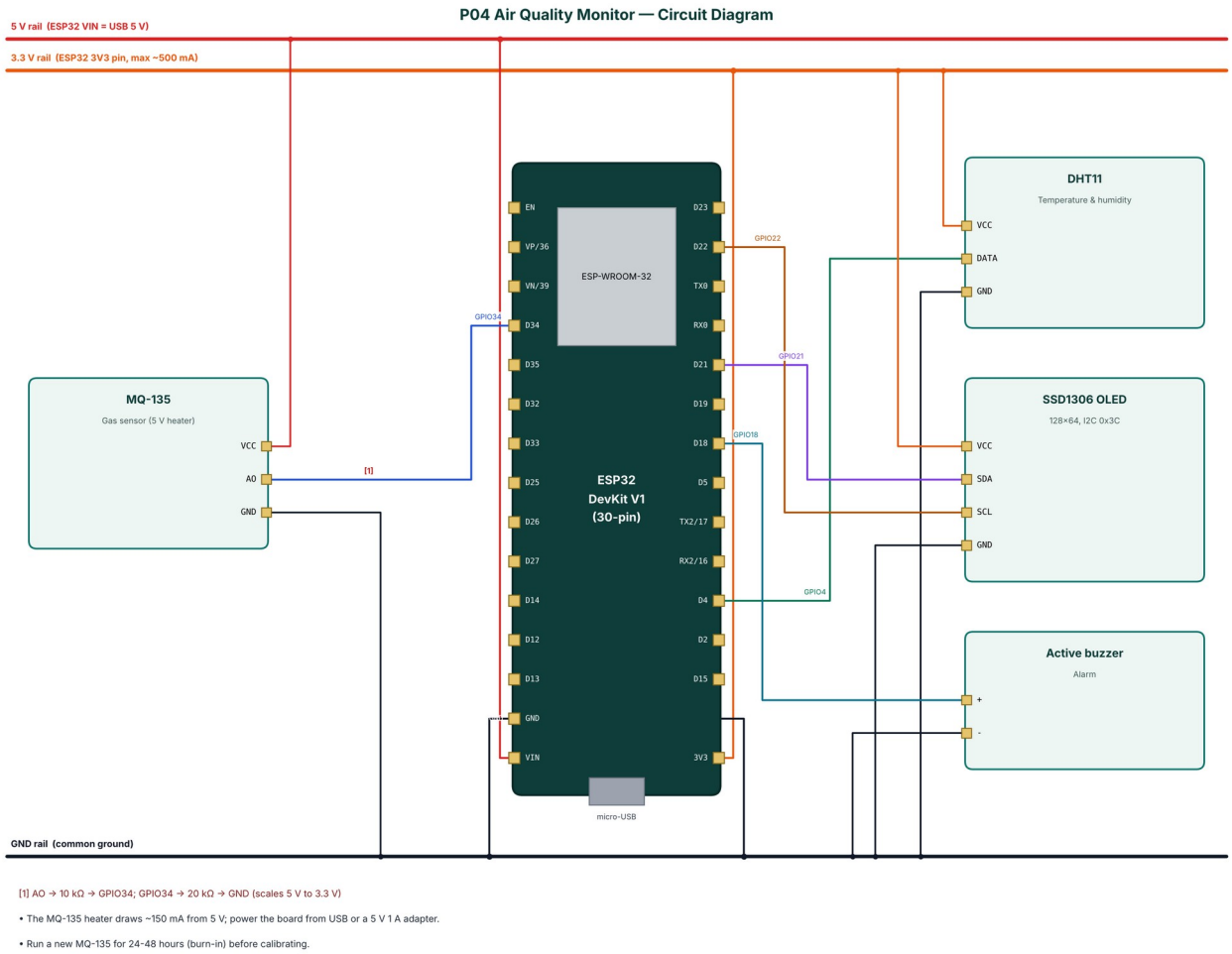


Figure 2: Circuit diagram of the Air Quality Monitor

4.2 Pin Connections

Module	Module pin	ESP32 connection	Notes
MQ-135	VCC	5V rail (VIN)	
MQ-135	AO	GPIO34	AO -> 10 kΩ -> GPIO34; GPIO34 -> 20 kΩ -> GND (scales 5 V to 3.3 V)
MQ-135	GND	GND	
DHT11	VCC	3.3 V (3V3 pin)	
DHT11	DATA	GPIO4	
DHT11	GND	GND	
SSD1306 OLED	VCC	3.3 V (3V3 pin)	
SSD1306 OLED	SDA	GPIO21	
SSD1306 OLED	SCL	GPIO22	
SSD1306 OLED	GND	GND	
Active buzzer	+	GPIO18	

Module	Module pin	ESP32 connection	Notes
Active buzzer	-	GND	

Table 3: Pin connection table

4.3 Interfacing and Power Notes

- The MQ-135 heater draws ~150 mA from 5 V; power the board from USB or a 5 V 1 A adapter.
- Run a new MQ-135 for 24-48 hours (burn-in) before calibrating.
- Only ADC1 pins (GPIO32-39) are used for analog inputs, because ADC2 cannot be read while Wi-Fi is active.
- Strapping pins GPIO0, 2, 12 and 15 are avoided for inputs so the board always boots normally.
- All grounds are common. 5 V signals are reduced to 3.3 V with resistor dividers before reaching the ESP32.

5. Data Flow Diagrams

Data flow diagrams (DFDs) show how data moves through the system without showing control logic or timing. External entities are drawn as rectangles, processes as numbered circles, data stores as open boxes and data flows as labelled arrows.

5.1 DFD Level 0 (Context Diagram)

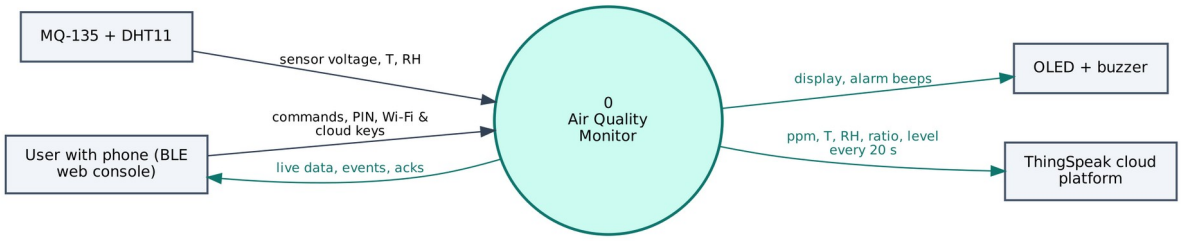


Figure 3: DFD Level 0 — context diagram

At level 0 the whole Air Quality Monitor is a single process. It receives sensor voltage, T, RH from the MQ-135 + DHT11, commands and configuration from the user's phone, and no commands from the cloud. It sends display, alarm beeps to the OLED + buzzer, live data to the phone and ppm, T, RH, ratio, level every 20 s to ThingSpeak.

5.2 DFD Level 1

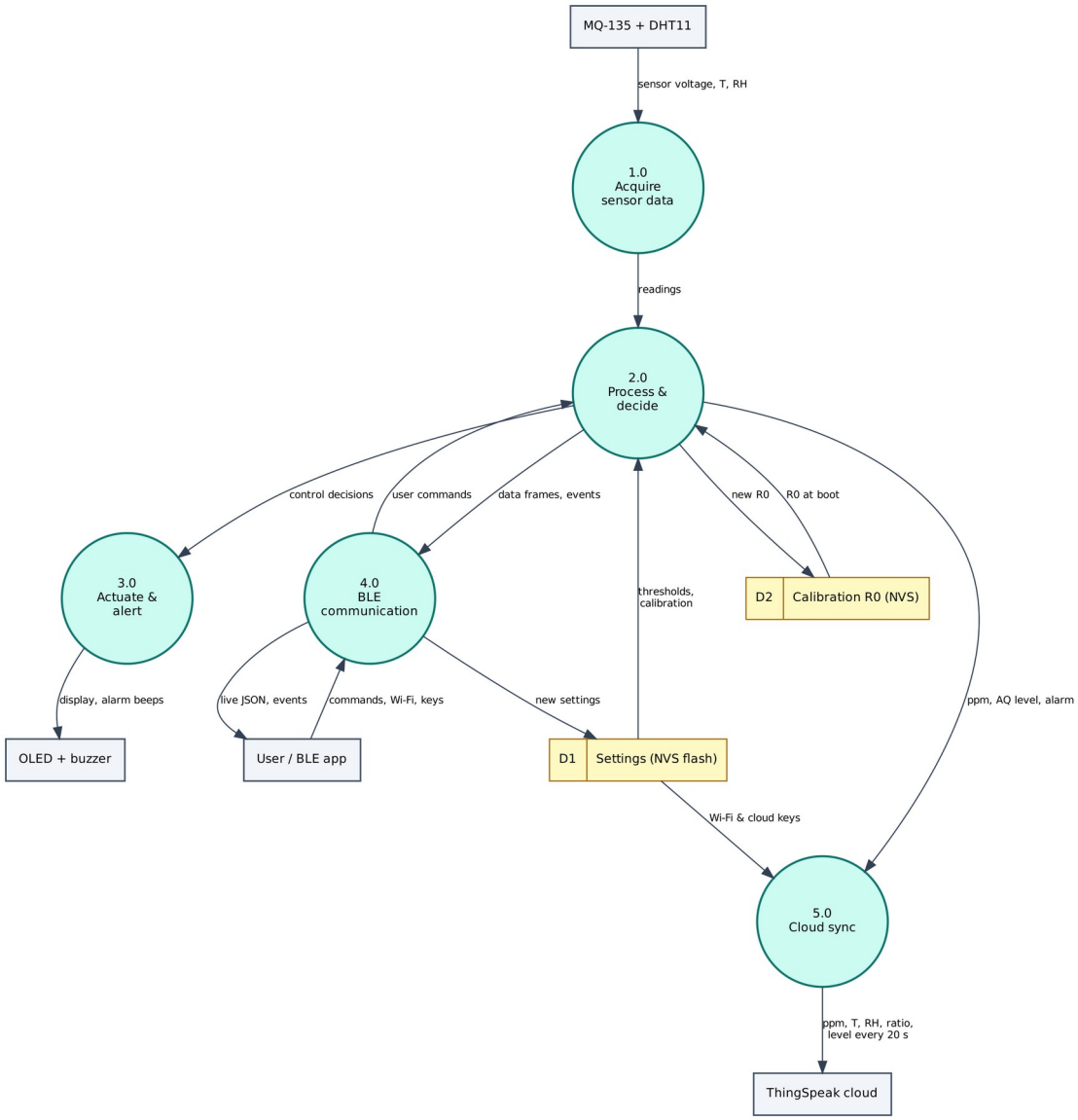


Figure 4: DFD Level 1 — main processes and data stores

Process / store	Description
1.0 Acquire sensor data	Reads sensor voltage, T, RH from the MQ-135 + DHT11; filters/averages the raw values.
2.0 Process & decide	Computes ppm, AQ level, alarm; applies thresholds and rules; generates events.
3.0 Actuate & alert	Drives the OLED + buzzer (display, alarm beeps).
4.0 BLE communication	Sends JSON frames and events to the phone; receives and authenticates commands.
5.0 Cloud sync	Sends ppm, T, RH, ratio, level every 20 s to ThingSpeak.
D1 Settings (NVS)	Wi-Fi credentials, cloud keys, PIN, thresholds — survive reboots.
D2 Calibration R0 (NVS)	Stores new R0; returns R0 at boot.

Table 4: Description of level-1 processes and data stores

5.3 Data Dictionary

Field	Meaning	Unit
ppm	CO_2-equivalent	ppm
cat	Level name	—
lvl	Level 0-5	—
t	Temperature	℃
h	Humidity	%
rs	Sensor resistance	kΩ
r0	Reference resistance	kΩ
ratio	Rs/R0	—
alarm	Alarm active	bool

Table 5: BLE data frame fields

Cloud object	Content / usage
Field 1	CO_2-equivalent (ppm)
Field 2	Temperature (℃)
Field 3	Humidity (%)
Field 4	Rs/R0
Field 5	Level (1-5)
Status	Level name

Table 6: ThingSpeak data objects

6. Software Design and Implementation

6.1 Development Environment

Item	Details
IDE / build	Arduino IDE 2.x or arduino-cli; GitHub Actions builds firmware in the cloud
Board package	esp32 by Espressif Systems 3.x (tested with 3.3.12)
Board settings	ESP32 Dev Module, Partition Scheme: Minimal SPIFFS (1.9MB APP with OTA/190KB SPIFFS)
Common libraries	NimBLE-Arduino 2.x, ArduinoJson 7.x, PubSubClient 2.8
Project libraries	DHT sensor library (Adafruit), Adafruit Unified Sensor, Adafruit SSD1306, Adafruit GFX Library
Phone app	Web Bluetooth console (Chrome on Android / desktop, Bluefy on iPhone); nRF Connect as alternative
Cloud	ThingSpeak (free tier)

Table 7: Software tools

> The default 1.2 MB app partition is too small for BLE + Wi-Fi + TLS. Select **Minimal SPIFFS (1.9MB APP)** in Tools → Partition Scheme, otherwise the upload fails with 'Sketch too big'.

6.2 Firmware Architecture

The firmware is split into three layers so that the ten projects share tested code:

- **IoTCore.h** (shared) — Wi-Fi station with auto-reconnect, NimBLE GATT server with the Nordic UART Service, line-based command parser shared by BLE and USB serial, PIN authentication, settings in NVS (Preferences), NTP time in IST, JSON helpers.
- **Cloud.h** (shared) — ThingSpeak HTTPS client, Adafruit IO MQTT/TLS client with a 28 messages/minute rate limiter, Supabase PostgREST client (insert, update, select).
- **P04_Air_Quality_Monitor.ino** — sensor drivers, algorithms, automation rules and the project's own commands.

The main loop is non-blocking: every task is scheduled with millis() timers, BLE writes are queued from the BLE callback into a FreeRTOS queue and executed in loop(), so the BLE stack never waits for slow operations such as HTTPS requests.

6.3 Working of the System

Every 2 seconds the firmware reads the DHT11 and averages 32 ADC readings of the MQ-135. The module voltage is recovered from the divider (×1.5) and Rs = RL·(Vc − Vout)/Vout. Rs is divided by the temperature/humidity correction factor and the ratio Rs/R0 is converted to ppm with the datasheet curve. An exponential moving average (α = 0.3) smooths the result.

Levels: Good < 600 ppm, Moderate < 1,000, Poor < 1,500, Unhealthy < 2,500, Hazardous above. When the value exceeds 'alarmppm' (default 1,500) the buzzer beeps and an 'air_alarm' event is sent; 'mute' silences it for 10 minutes. The calibration command averages 20 readings in clean air and sets R0 = Rs·(420/116.6)^(1/2.769), so clean air reads 420 ppm.

Every 20 seconds after warm-up, ppm, temperature, humidity, Rs/R0 and level are uploaded to ThingSpeak with the level name as status text.

6.4 Flowchart

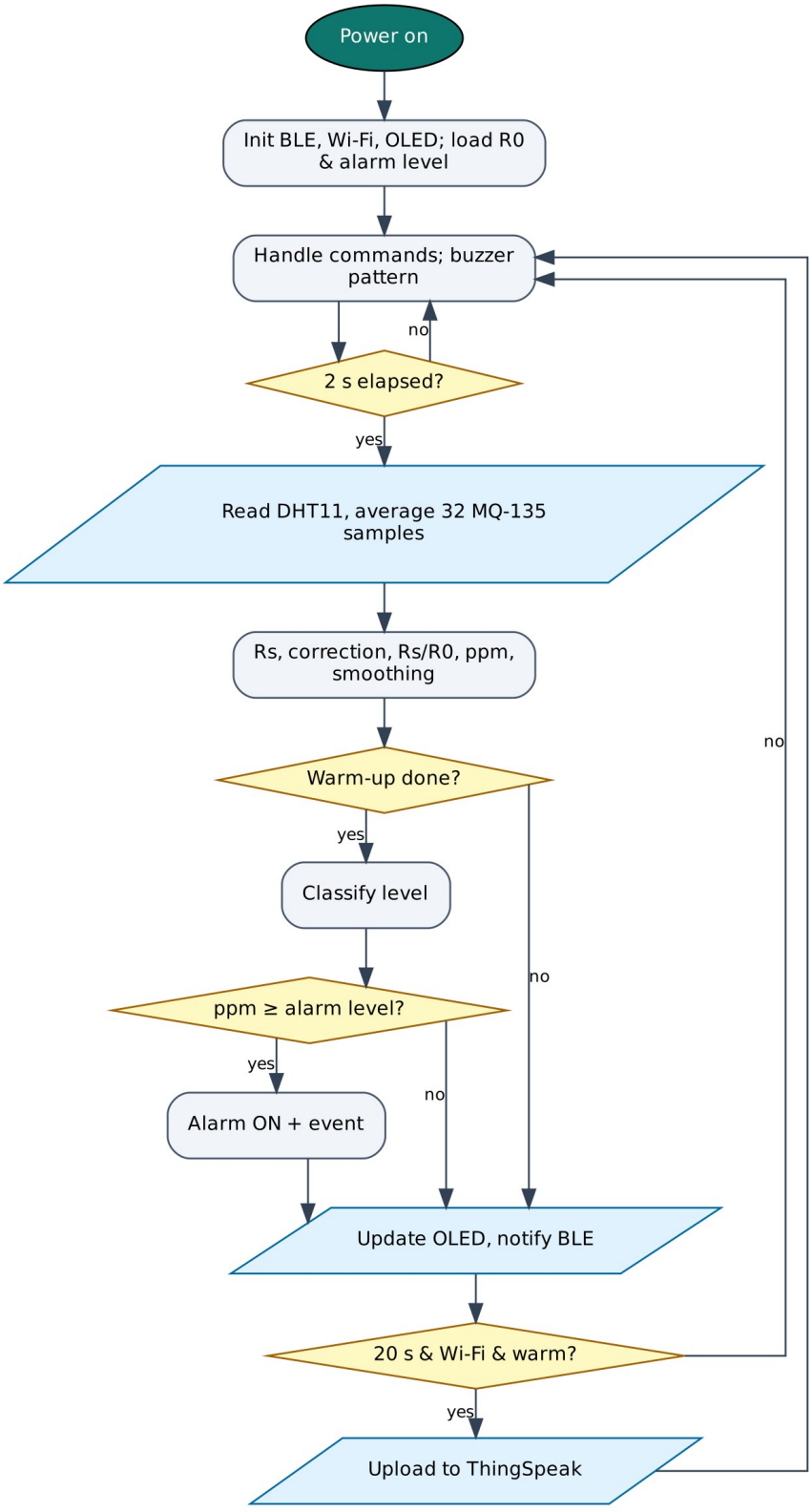


Figure 5: Firmware flowchart

6.5 BLE Communication Protocol

The device is a BLE peripheral named **IOT-P04-XXXX** (XXXX = last bytes of the MAC address). It exposes the Nordic UART Service, which most BLE apps understand:

Item	UUID	Properties
Service (NUS)	6E400001-B5A3-F393-E0A9-E50E24DCCA9E	Primary
RX — phone → ESP32	6E400002-B5A3-F393-E0A9-E50E24DCCA9E	Write, Write without response
TX — ESP32 → phone	6E400003-B5A3-F393-E0A9-E50E24DCCA9E	Notify, Read

Table 8: GATT service and characteristics

Commands are plain text lines (`command:argument`). Replies are one JSON object per line ending in a newline; long lines are split into MTU-sized notifications and re-joined by the app. Commands that change anything require `auth:<PIN>` first on each connection (default PIN 1234, change it with `set:pin=XXXX`).

Command	PIN needed	Action
auth:<PIN>	—	Unlock for this connection
status	No	Status JSON: Wi-Fi, IP, RSSI, heap, uptime, time
wifi:<SSID>,<password>	Yes	Save Wi-Fi credentials and connect
set:<key>=<value>	Yes	Save a setting (cloud keys, thresholds)
get:<key>	Yes	Read a setting (secrets masked)
reboot / forget	Yes	Restart / erase all settings
calibrate	Yes	Set R0 in clean outdoor air
mute	No	Mute alarm for 10 minutes

Table 9: BLE commands (common commands first, then project commands)

Example data frame sent by the device:

```
{"ppm":684,"t":27.9,"h":55,"rs":38.12,"r0":52.4,"ratio":0.727,"lvl":2,"cat":"Moderate","alarm":false,"type":"data"}
```

6.6 Cloud Integration (ThingSpeak)

Data is sent with an HTTPS GET request: `https://api.thingspeak.com/update?api_key=KEY&field1=…&status=….` ThingSpeak returns the new entry number, or 0 if the update was rejected (for example when sent within 15 s of the previous one). The free licence allows 3 million messages per year and one update every 15 s per channel, so a 20 s interval leaves a safety margin.

Object	Content
Field 1	CO_2-equivalent (ppm)
Field 2	Temperature (°C)
Field 3	Humidity (%)
Field 4	Rs/R0
Field 5	Level (1-5)
Status	Level name

Table 10: ThingSpeak configuration

6.7 Configurable Settings

Key	Default	Meaning
tskey	—	ThingSpeak Write API Key
alarmppm	1500	Alarm threshold (ppm)
r0	10	Reference resistance (set by calibrate)
ssid / pass	—	Wi-Fi network (set with the wifi: command)
pin	1234	BLE PIN

Table 11: Settings stored in flash (change with set:key=value)

7. Setup and Deployment Guide

7.1 Hardware Assembly

1. Place the ESP32 DevKit across the centre gap of the breadboard so both pin rows are free.
2. Connect the ESP32 3V3 pin to one breadboard power rail (3.3 V), VIN to the other rail (5 V) and both GND pins to the ground rails.
3. Wire each module exactly as in the pin connection table (section 4.2), including the resistor dividers marked with [n] in the circuit diagram.
4. Check every connection against the circuit diagram before powering on. Make sure no 5 V signal reaches an ESP32 pin directly.
5. Connect the USB cable to a computer (for flashing) or a 5 V adapter (for normal use).

7.2 Flashing the Firmware

Option A — Arduino IDE

1. Install Arduino IDE 2.x from arduino.cc.
2. File → Preferences → Additional boards manager URLs: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`.
3. Tools → Board → Boards Manager: install **esp32 by Espressif Systems** (version 3.x).
4. Sketch → Include Library → Manage Libraries: install **NimBLE-Arduino** (2.x), **ArduinoJson** (7.x), **PubSubClient**, **DHT sensor library (Adafruit)**, **Adafruit Unified Sensor**, **Adafruit SSD1306**, **Adafruit GFX Library**.
5. Download the repository and open the folder **P04_Air_Quality_Monitor** (it already contains IoTCore.h, Cloud.h and config.h).
6. Tools: Board **ESP32 Dev Module**, Partition Scheme **Minimal SPIFFS (1.9MB APP with OTA/190KB SPIFFS)**, Upload Speed 921600, and the correct COM port.
7. Click Upload. If it stays at 'Connecting...', hold the BOOT button on the ESP32 until upload starts.
8. Open Serial Monitor at 115200 baud and type `status` to check the device.

Option B — no software installation (GitHub Actions + browser flasher)

1. Open the repository https://github.com/rohitkukreja705/esp32-iot-projects → **Actions** → latest **Build firmware** run.
2. Download the artifact **firmware-P04_Air_Quality_Monitor** and unzip it to get `P04_Air_Quality_Monitor.ino.merged.bin`.
3. Open **https://espressif.github.io/esptool-js/** in Chrome or Edge on a computer, click **Connect** and select the ESP32's serial port.
4. Set Flash Address **0x0**, choose the merged .bin file and click **Program**. Press the EN (reset) button when done.

7.3 Cloud Setup (ThingSpeak)

1. Go to **thingspeak.mathworks.com** and sign up with a free MathWorks account (choose the free, non-commercial licence).
2. Open **Channels → My Channels → New Channel**. Name it 'Air Quality Monitor'.
3. Tick and name the fields exactly as listed in the table below, then click **Save Channel**.
4. Open the **API Keys** tab and copy the **Write API Key** (16 characters).
5. In the BLE web console open **Setup**, unlock with the PIN and save the key in the `tskey` field (equivalent command: `set:tskey=YOURKEY`).
6. Open **Private View** to see live charts. Add Gauge or Numeric Display widgets with **Add Widgets**, and use **Sharing → Share channel view with everyone** if you want a public link.

Field / feed / table	Content
Field 1	CO_2-equivalent (ppm)
Field 2	Temperature (°C)
Field 3	Humidity (%)
Field 4	Rs/R0
Field 5	Level (1-5)
Status	Level name

Table 12: Items to create in ThingSpeak

7.4 BLE App Setup (Web Bluetooth Console)

No app installation is needed. The console is a web page hosted free on GitHub Pages; Chrome talks to the ESP32 directly through the Web Bluetooth API.

1. On an Android phone turn on **Bluetooth** and **Location** (Android requires location for BLE scanning).
2. Open **https://rohitkukreja705.github.io/esp32-iot-projects/?p=P04** in **Chrome**. (On iPhone use the free **Bluefy** browser; on a laptop use Chrome or Edge.)
3. Tap **Connect** and choose **IOT-P04-XXXX** in the pop-up list. The dot turns green and the Live tab starts updating.
4. Open **Setup**, enter the PIN (default **1234**) and tap **Unlock**.
5. Enter your 2.4 GHz Wi-Fi name and password and tap **Save Wi-Fi**. After a few seconds the status line shows the IP address.
6. Enter the ThingSpeak keys from section 7.3 and tap **Save** next to each.
7. Use the **Control** tab for commands and the **Log** tab to see every event and acknowledgement.

Alternative: nRF Connect app

1. Install **nRF Connect for Mobile** (Nordic Semiconductor, free) and scan.
2. Connect to **IOT-P04-XXXX** and open **Nordic UART Service**.
3. Enable notifications on the **TX** characteristic (6E400003…).
4. Write UTF-8 text to the **RX** characteristic (6E400002…), e.g. `auth:1234`, then `status`.

> To host the console yourself: in the GitHub repository open Settings → Pages, choose 'Deploy from a branch', branch main, folder /docs. Web Bluetooth only works on HTTPS pages, which GitHub Pages provides.

7.5 First-Run Checklist

- Serial Monitor shows '[BLE] advertising as IOT-P04-XXXX'.
- Status in the console shows wifi: true and an IP address.
- Data appears in ThingSpeak within a minute.
- All sensor values in the Live tab are plausible (no '—').

8. Testing and Results

The following test cases verify each function. Record the observed result and mark Pass/Fail while testing your build.

#	Test	Procedure	Expected result	Observed
T1	Warm-up	Power on and watch the OLED	'Warm-up' for 2 minutes, then a ppm value	
T2	Calibration	Place outdoors for 10 min, send calibrate	Reading becomes ≈ 420 ppm; new R0 reported	
T3	Breath test	Breathe gently towards the sensor	ppm rises within seconds, then falls back	
T4	Alcohol test	Hold a hand-sanitiser cap 10 cm away	Level rises to Poor/Unhealthy; alarm sounds above threshold	
T5	Mute	Send mute during an alarm	Buzzer stops; OLED shows '(muted)'	
T6	Cloud	Check ThingSpeak after 2 minutes	Five fields updated every 20 s	

Table 13: Test cases

8.1 Expected Output

A typical data frame received in the console's Log tab looks like this (values will differ):

```
{"ppm":684,"t":27.9,"h":55,"rs":38.12,"r0":52.4,"ratio":0.727,"lvl":2,"cat":"Moderate","alarm":false,"type":"data"}
```

Serial Monitor (115200 baud) prints the same lines along with connection messages such as `[WiFi] connected`, `[BLE] connected` and the HTTP or MQTT result of each cloud upload, which helps in troubleshooting.

8.2 Troubleshooting

Symptom	Likely cause and fix
Device not in the Connect list	Bluetooth/Location off on the phone, or the page is not opened in Chrome over HTTPS.
'Sketch too big' on upload	Select Partition Scheme 'Minimal SPIFFS (1.9MB APP with OTA)'.
Wi-Fi never connects	ESP32 supports only 2.4 GHz networks; check the SSID/password (case-sensitive).
Settings refused	Send auth:<PIN> first; the unlock is per BLE connection.
Sensor shows null / —	Check wiring and supply voltage; see the Serial Monitor for 'not found' messages.
Board resets randomly	Weak USB supply or servo/relay current; use a 5 V 2 A adapter and add a 470 µF capacitor.
No data in ThingSpeak	Wrong Write API Key or interval below 15 s.

Table 14: Troubleshooting guide

9. Discussion

9.1 Advantages

- Very low cost compared with NDIR CO_2 meters.
- Temperature/humidity compensation and calibration improve consistency.
- Local display, phone alerts and cloud history.
- Thresholds adjustable without reprogramming.

9.2 Limitations

- MQ-135 is not selective: alcohol, ammonia and smoke also raise the 'CO_2' value.
- Values are indicative; accuracy is far below an NDIR sensor (e.g. MH-Z19, SCD40).
- Needs burn-in and 2-minute warm-up; heater consumes ~0.75 W.
- Not certified for LPG leak safety.

9.3 Safety Precautions

- The MQ-135 heater gets hot (~50 °C on the case); do not touch it during operation.
- Do not rely on this device for life-safety gas detection.

9.4 Security Considerations

- Secrets (Wi-Fi password, cloud keys) are stored in the ESP32's NVS flash, not in source code.
- BLE commands that change the device require a PIN; change the default PIN after setup.
- Cloud traffic is TLS-encrypted. Certificate verification is disabled (setInsecure) for simplicity; for production, pin the server's root certificate with setCACert().
- Use a dedicated cloud account/project for the device and rotate keys if the device is lost.

9.5 Future Scope

- Add a PM2.5 sensor (PMS5003/SDS011) and an NDIR CO_2 sensor for a full AQI.
- Automatically switch an exhaust fan through a relay.
- Use ThingSpeak React to send alerts when the level stays Poor for 30 minutes.
- Battery operation with heater duty-cycling.

10. Conclusion

The monitor turns a cheap metal-oxide sensor into a practical air-quality indicator by adding calibration, environmental compensation and smoothing. With a local display, BLE alerts and ThingSpeak logging it helps users decide when to ventilate, and its modular design allows more accurate sensors to be added later.

All design goals listed in Chapter 1 are met with low-cost parts available in the Indian market, free cloud services and open-source software. The shared firmware core and the single Web Bluetooth console make the project easy to reproduce and extend.

References

1. Hanwei Electronics, MQ-135 Gas Sensor technical data.
2. G. Krocker, MQ135 Arduino library (correction-factor coefficients), GitHub.
3. NOAA Global Monitoring Laboratory, Trends in atmospheric CO_2 (global mean ≈ 420 ppm).
4. Solomon Systech, SSD1306 OLED driver datasheet.
5. Espressif Systems, ESP32 Series Datasheet and ESP32 Technical Reference Manual, espressif.com.
6. Espressif Systems, Arduino-ESP32 documentation, docs.espressif.com/projects/arduino-esp32.
7. H2zero, NimBLE-Arduino library documentation, github.com/h2zero/NimBLE-Arduino.
8. Bluetooth SIG, Bluetooth Core Specification 4.2 — Generic Attribute Profile (GATT).
9. W3C Web Bluetooth Community Group, Web Bluetooth specification, webbluetoothcg.github.io/web-bluetooth.
10. B. Blanchon, ArduinoJson 7 documentation, arduinojson.org.
11. MathWorks, ThingSpeak documentation — Write data with REST API, mathworks.com/help/thingspeak.

Appendix A: Project Source Code

The complete code is in the repository folder **P04_Air_Quality_Monitor** (https://github.com/rohitkukreja705/esp32-iot-projects).

config.h

```
#pragma once
// ===== P04 Air Quality Monitor — configuration =====
#define WIFI_SSID     ""
#define WIFI_PASS     ""
#define DEVICE_PIN    "1234"
#define TS_WRITE_KEY  ""

// ----- pins -----
#define PIN_MQ135     34     // MQ-135 AO through 10k/20k divider (5 V -> 3.3 V)
#define PIN_DHT       4
#define PIN_BUZZER    18
// SSD1306 128x64 OLED: SDA 21, SCL 22, address 0x3C
#define OLED_ADDR     0x3C

#define DIVIDER_RATIO 1.5f   // module AO = ADC voltage x 1.5 (10k top, 20k bottom)
#define MQ_VCC_MV     5000.0f

// ----- behaviour -----
#define WARMUP_S        120   // heater warm-up after each power-on
#define DEF_ALARM_PPM   1500  // set:alarmppm=...
#define SAMPLE_MS       2000
#define TS_INTERVAL_S   20
```

P04_Air_Quality_Monitor.ino

```
/*
 * P04 — Indoor Air Quality Monitor with Alarm
 * ===========================================
 * Peripherals: MQ-135 gas sensor, DHT11, 0.96" SSD1306 OLED, active buzzer
 * Wi-Fi  : ThingSpeak — ppm, temperature, humidity, Rs/R0, level every 20 s
 * BLE    : live values, one-tap clean-air calibration, alarm mute
 * Extras : temperature/humidity compensation of the MQ-135 resistance,
 *          exponential smoothing, 5-level air quality classification,
 *          warm-up handling, R0 calibration saved to flash
 *
 * NOTE: MQ-135 gives an *indicative* CO2-equivalent value (it also reacts to
 * NH3, alcohol, smoke, benzene). It is not a certified AQI instrument.
 */
#include "config.h"
#include "IoTCore.h"
#include "Cloud.h"
#include <Wire.h>
#include <DHT.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

DHT dht(PIN_DHT, DHT11);
Adafruit_SSD1306 oled(128, 64, &Wire, -1);
bool oledOK = false;

// MQ-135 curve for CO2 (from datasheet log-log fit)
const float PARA = 116.6020682, PARB = 2.769034857;
const float ATMOCO2 = 420.0;                        // outdoor CO2 today, ppm
```

```
float temp = NAN, hum = NAN, rs = NAN, r0 = 10.0, ratio = NAN, ppm = NAN;
int level = 0; bool muted = false, alarmOn = false;
uint32_t lastSample = 0, lastUpload = 0, alarmMuteUntil = 0;
float alarmPpm = DEF_ALARM_PPM;
const char* LEVELS[] = { "Warming up", "Good", "Moderate", "Poor", "Unhealthy", "Hazardous" };

// Correction factor for temperature and humidity (MQ-135 datasheet curves)
float correction(float t, float h) {
  if (isnan(t) || isnan(h)) return 1.0;
  if (t < 20) return 0.00035 * t * t - 0.02718 * t + 1.39538 - (h - 33.) * 0.0018;
  return -0.003333333 * t + -0.001923077 * h + 1.130128205;
}

// Sensor resistance in kOhm. RL value cancels out in Rs/R0, so 10 k is used.
float readRs() {
  uint32_t s = 0;
  for (int i = 0; i < 32; i++) { s += analogReadMilliVolts(PIN_MQ135); delayMicroseconds(300); }
  float vout = (s / 32.0) * DIVIDER_RATIO;
  if (vout < 1) vout = 1;
  float rsRaw = 10.0 * (MQ_VCC_MV - vout) / vout;
  return rsRaw / correction(temp, hum);
}

int classify(float p) {
  if (millis() < WARMUP_S * 1000UL) return 0;
  if (p < 600) return 1;
  if (p < 1000) return 2;
  if (p < 1500) return 3;
  if (p < 2500) return 4;
  return 5;
}

void sample() {
  float t = dht.readTemperature(), h = dht.readHumidity();
  if (!isnan(t)) temp = t;
  if (!isnan(h)) hum = h;
  rs = readRs();
  ratio = rs / r0;
  float p = PARA * pow(ratio, -PARB);
  ppm = isnan(ppm) ? p : ppm * 0.7 + p * 0.3;      // exponential smoothing
  level = classify(ppm);

  bool alarmNow = level > 0 && ppm >= alarmPpm;
  if (alarmNow && !alarmOn) iot::sendEvent("air_alarm", "Air quality " + String(LEVELS[level]) + ", " +
String(ppm, 0) + " ppm");
  if (!alarmNow && alarmOn) iot::sendEvent("air_ok", "Air quality back to normal");
  alarmOn = alarmNow;
}

void drawOled() {
  if (!oledOK) return;
  oled.clearDisplay();
  oled.setTextColor(SSD1306_WHITE);
  oled.setTextSize(1); oled.setCursor(0, 0);
  oled.print("AIR QUALITY  ");
  oled.print(iot::wifiOK() ? "W" : "-"); oled.print(iot::bleConnected() ? " B" : " -");
  oled.setTextSize(3); oled.setCursor(0, 14);
  if (level == 0) { oled.setTextSize(2); oled.print("Warm-up"); }
  else oled.print((int)ppm);
  oled.setTextSize(1); oled.setCursor(98, 28); if (level) oled.print("ppm");
  oled.setCursor(0, 42); oled.print(LEVELS[level]);
  if (alarmOn) oled.print(muted ? " (muted)" : " !ALARM");
  oled.setCursor(0, 54);
```

```
  oled.printf("%.1fC  %.0f%%RH  R%.2f", temp, hum, ratio);
  oled.display();
}

void buzzer() {
  bool sound = alarmOn && millis() > alarmMuteUntil;
  digitalWrite(PIN_BUZZER, sound && (millis() / 250) % 4 == 0);   // short beeps
  muted = alarmOn && !sound;
}

void sendBle() {
  JsonDocument d;
  d["ppm"] = iot::rnd(ppm, 0); d["t"] = iot::rnd(temp); d["h"] = iot::rnd(hum, 0);
  d["rs"] = iot::rnd(rs, 2); d["r0"] = iot::rnd(r0, 2); d["ratio"] = iot::rnd(ratio, 3);
  d["lvl"] = level; d["cat"] = LEVELS[level]; d["alarm"] = alarmOn;
  iot::sendData(d);
}

// Commands: calibrate (in clean outdoor air), mute
bool onCommand(const String& cmd, const String& arg, bool authed) {
  if (cmd == "setting") { alarmPpm = iot::settingF("alarmppm", DEF_ALARM_PPM); return true; }
  if (cmd == "mute") { alarmMuteUntil = millis() + 10 * 60000UL; iot::emitAck(cmd, true, "muted 10
min"); return true; }
  if (cmd == "calibrate") {
    if (!authed) { iot::emitAck(cmd, false, "send auth:<PIN> first"); return true; }
    float sum = 0;
    for (int i = 0; i < 20; i++) { sum += readRs(); delay(100); }
    float rsAvg = sum / 20;
    r0 = rsAvg * pow(ATMOCO2 / PARA, 1.0 / PARB);    // R0 such that clean air = ATMOCO2 ppm
    iot::saveSetting("r0", String(r0, 3));
    ppm = NAN;
    iot::emitAck(cmd, true, "R0=" + String(r0, 2) + " kOhm");
    return true;
  }
  return false;
}

void setup() {
  pinMode(PIN_BUZZER, OUTPUT);
  analogSetAttenuation(ADC_11db);
  iot::begin("P04", onCommand);
  r0 = iot::settingF("r0", 10.0);
  alarmPpm = iot::settingF("alarmppm", DEF_ALARM_PPM);
  dht.begin();
  Wire.begin(21, 22);
  oledOK = oled.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR);
  if (!oledOK) Serial.println("[OLED] not found");
}

void loop() {
  iot::loop();
  buzzer();
  uint32_t now = millis();
  if (now - lastSample >= SAMPLE_MS) {
    lastSample = now;
    sample();
    drawOled();
    sendBle();
  }
  if (iot::wifiOK() && level > 0 && now - lastUpload >= TS_INTERVAL_S * 1000UL) {
    lastUpload = now;
    float f[5] = { ppm, temp, hum, ratio, (float)level };
    cloud::thingSpeakUpdate(f, 5, LEVELS[level]);
```

```
  }
}
```

Appendix B: Shared Library Code

These two headers are identical in every project folder (master copies in /common).

IoTCore.h

```
/*
 * IoTCore.h  —  shared runtime for all ESP32 IoT projects in this repo
 * ---------------------------------------------------------------------
 *  - Wi-Fi station with automatic reconnect (credentials stored in flash)
 *  - BLE server using the Nordic UART Service (NUS) layout, so the device
 *    works with the Web Bluetooth dashboard in /docs AND with generic apps
 *    such as "nRF Connect" or "Serial Bluetooth Terminal"
 *  - Line-based command protocol shared by BLE and USB Serial
 *  - Persistent settings (Preferences / NVS): Wi-Fi, cloud keys, PIN, thresholds
 *  - NTP time sync (IST) for timestamps
 *
 *  Wire protocol
 *  -------------
 *  Phone -> ESP32 (write to RX characteristic, text):
 *      auth:1234              unlock control + settings for this connection
 *      status                 device replies with a status JSON line
 *      set:key=value          store a setting (Wi-Fi, cloud key, threshold…)
 *      get:key                read back a setting (secrets are masked)
 *      wifi:SSID,PASSWORD     shortcut for set:ssid + set:pass + reconnect
 *      reboot                 restart the board
 *      <anything else>        forwarded to the project's own command handler
 *  ESP32 -> Phone (notify on TX characteristic):
 *      one JSON object per line, terminated by '\n'
 *      {"type":"data",...}  {"type":"status",...}  {"type":"ack",...}
 *      {"type":"event",...} {"type":"err",...}
 *  Lines longer than one BLE packet are split into chunks; the receiver
 *  joins chunks until it sees '\n'.
 *
 *  Library: NimBLE-Arduino 2.x (lighter than Bluedroid, leaves more RAM for TLS)
 *  Board:   ESP32 Dev Module, Partition Scheme "Minimal SPIFFS (1.9MB APP with OTA)"
 */
#pragma once

#include <Arduino.h>
#include <WiFi.h>
#include <Preferences.h>
#include <NimBLEDevice.h>
#include <ArduinoJson.h>
#include <time.h>

#ifndef DEVICE_PIN
#define DEVICE_PIN "1234"
#endif
#ifndef WIFI_SSID
#define WIFI_SSID ""
#endif
#ifndef WIFI_PASS
#define WIFI_PASS ""
#endif

namespace iot {

// Nordic UART Service UUIDs (widely supported by BLE terminal apps)
static const char* NUS_SERVICE = "6E400001-B5A3-F393-E0A9-E50E24DCCA9E";
static const char* NUS_RX      = "6E400002-B5A3-F393-E0A9-E50E24DCCA9E";  // phone writes here
static const char* NUS_TX      = "6E400003-B5A3-F393-E0A9-E50E24DCCA9E";  // ESP32 notifies here
```

```
// Project command handler: return true if the command was recognised.
using CommandHandler = bool (*)(const String& cmd, const String& arg, bool authed);

// ---------------------------------------------------------------- state
struct CmdMsg { char text[256]; bool fromBle; };

static Preferences        prefs;
static NimBLEServer*      bleServer   = nullptr;
static NimBLECharacteristic* txChr    = nullptr;
static QueueHandle_t      cmdQueue    = nullptr;
static CommandHandler     userHandler = nullptr;
static String             deviceName;
static String             projectId;
static volatile uint16_t  bleMtu      = 23;
static volatile bool      bleAuthed   = false;   // reset on every new connection
static bool               serialAuthed = true;   // USB = physical access, trusted
static bool               timeSynced  = false;
static uint32_t           lastWifiTry = 0;
static String             rxLine;                // assembles multi-packet writes

// ---------------------------------------------------------------- settings
inline String setting(const char* key, const char* def = "") {
  return prefs.isKey(key) ? prefs.getString(key, def) : String(def);
}
inline void saveSetting(const char* key, const String& v) { prefs.putString(key, v); }
inline float settingF(const char* key, float def) {
  String s = setting(key, "");
  return s.length() ? s.toFloat() : def;
}

// ---------------------------------------------------------------- BLE
inline bool bleConnected() { return bleServer && bleServer->getConnectedCount() > 0; }

// Send one line (adds '\n'), split to fit the negotiated MTU.
inline void bleSend(const String& line) {
  if (!bleConnected() || !txChr) return;
  String msg = line + "\n";
  size_t chunk = (bleMtu > 23 ? bleMtu : 23) - 3;
  for (size_t i = 0; i < msg.length(); i += chunk) {
    size_t n = min(chunk, msg.length() - i);
    txChr->setValue((const uint8_t*)msg.c_str() + i, n);
    txChr->notify();
    if (msg.length() > chunk) delay(8);   // give the stack time between chunks
  }
}

// Send to BLE and mirror to USB Serial.
inline void emit(const String& line) {
  Serial.println(line);
  bleSend(line);
}

inline void emitAck(const String& cmd, bool ok, const String& msg = "") {
  JsonDocument d;
  d["type"] = ok ? "ack" : "err";
  d["cmd"] = cmd;
  if (msg.length()) d["msg"] = msg;
  String s; serializeJson(d, s); emit(s);
}

inline void enqueue(const String& s, bool fromBle) {
  if (!cmdQueue || s.length() == 0) return;
```

```
  CmdMsg m; memset(&m, 0, sizeof(m));
  strncpy(m.text, s.c_str(), sizeof(m.text) - 1);
  m.fromBle = fromBle;
  xQueueSend(cmdQueue, &m, 0);
}

class ServerCB : public NimBLEServerCallbacks {
  void onConnect(NimBLEServer* s, NimBLEConnInfo& info) override {
    bleAuthed = false;
    rxLine = "";
    Serial.printf("[BLE] connected %s\n", info.getAddress().toString().c_str());
  }
  void onDisconnect(NimBLEServer* s, NimBLEConnInfo& info, int reason) override {
    bleAuthed = false;
    bleMtu = 23;
    Serial.printf("[BLE] disconnected (reason %d), advertising again\n", reason);
    // NimBLE restarts advertising automatically (advertiseOnDisconnect = true)
  }
  void onMTUChange(uint16_t mtu, NimBLEConnInfo& info) override {
    bleMtu = mtu;
    Serial.printf("[BLE] MTU %u\n", mtu);
  }
};

class RxCB : public NimBLECharacteristicCallbacks {
  void onWrite(NimBLECharacteristic* c, NimBLEConnInfo& info) override {
    std::string v = c->getValue();
    // Accept either a single write without newline, or several writes
    // that together form lines terminated by '\n'.
    for (char ch : v) {
      if (ch == '\r') continue;
      if (ch == '\n') { enqueue(rxLine, true); rxLine = ""; }
      else if (rxLine.length() < 250) rxLine += ch;
    }
    if (v.find('\n') == std::string::npos && rxLine.length() && rxLine.length() == v.length()) {
      enqueue(rxLine, true);  // plain one-shot write (e.g. typed in nRF Connect)
      rxLine = "";
    }
  }
};

inline void bleBegin() {
  uint8_t mac[6]; WiFi.macAddress(mac);
  char suffix[6]; snprintf(suffix, sizeof(suffix), "%02X%02X", mac[4], mac[5]);
  deviceName = "IOT-" + projectId + "-" + suffix;

  NimBLEDevice::init(deviceName.c_str());
  NimBLEDevice::setMTU(247);
  bleServer = NimBLEDevice::createServer();
  bleServer->setCallbacks(new ServerCB());

  NimBLEService* svc = bleServer->createService(NUS_SERVICE);
  txChr = svc->createCharacteristic(NUS_TX, NIMBLE_PROPERTY::NOTIFY | NIMBLE_PROPERTY::READ);
  NimBLECharacteristic* rx = svc->createCharacteristic(
      NUS_RX, NIMBLE_PROPERTY::WRITE | NIMBLE_PROPERTY::WRITE_NR, 512);
  rx->setCallbacks(new RxCB());
  // (NimBLE 2.x starts services automatically when advertising begins)

  // 128-bit UUID goes in the advertisement, the name in the scan response
  // (both together would not fit in one 31-byte packet).
  NimBLEAdvertisementData adv, scan;
  adv.setFlags(BLE_HS_ADV_F_DISC_GEN | BLE_HS_ADV_F_BREDR_UNSUP);
  adv.setCompleteServices(NimBLEUUID(NUS_SERVICE));
```

```
  scan.setName(deviceName.c_str());
  NimBLEAdvertising* a = NimBLEDevice::getAdvertising();
  a->setAdvertisementData(adv);
  a->setScanResponseData(scan);
  a->start();
  Serial.printf("[BLE] advertising as %s\n", deviceName.c_str());
}

// ---------------------------------------------------------------- Wi-Fi
inline bool wifiOK() { return WiFi.status() == WL_CONNECTED; }

inline void wifiConnect() {
  String ssid = setting("ssid", WIFI_SSID);
  String pass = setting("pass", WIFI_PASS);
  if (ssid.isEmpty()) {
    Serial.println("[WiFi] no SSID yet - send  wifi:SSID,PASSWORD  over BLE");
    return;
  }
  Serial.printf("[WiFi] connecting to %s\n", ssid.c_str());
  WiFi.disconnect();
  WiFi.begin(ssid.c_str(), pass.c_str());
  lastWifiTry = millis();
}

inline void wifiLoop() {
  static bool wasUp = false;
  bool up = wifiOK();
  if (up && !wasUp) {
    Serial.printf("[WiFi] connected, IP %s, RSSI %d\n",
                  WiFi.localIP().toString().c_str(), WiFi.RSSI());
    configTzTime("IST-5:30", "pool.ntp.org", "time.google.com");
  }
  if (!up && millis() - lastWifiTry > 20000) wifiConnect();   // retry every 20 s
  wasUp = up;
  if (up && !timeSynced && time(nullptr) > 1700000000) {
    timeSynced = true;
    Serial.println("[NTP] time synced");
  }
}

// ISO-8601 local time, or uptime if NTP has not synced yet.
inline String timestamp() {
  time_t now = time(nullptr);
  if (now < 1700000000) return "uptime+" + String(millis() / 1000) + "s";
  struct tm t; localtime_r(&now, &t);
  char buf[32]; strftime(buf, sizeof(buf), "%Y-%m-%dT%H:%M:%S+05:30", &t);
  return String(buf);
}

// ------------------------------------------------------- built-in commands
inline bool isSecretKey(const String& k) {
  return k == "pass" || k.endsWith("key") || k == "pin";
}

inline void sendStatus() {
  JsonDocument d;
  d["type"] = "status";
  d["dev"] = deviceName;
  d["wifi"] = wifiOK();
  d["ssid"] = setting("ssid", WIFI_SSID);
  d["rssi"] = wifiOK() ? WiFi.RSSI() : 0;
  d["ip"] = wifiOK() ? WiFi.localIP().toString() : "";
  d["auth"] = bleAuthed;
```

```
  d["heap"] = ESP.getFreeHeap();
  d["up"] = millis() / 1000;
  d["time"] = timestamp();
  String s; serializeJson(d, s); emit(s);
}

inline void handleLine(const String& raw, bool fromBle) {
  String line = raw; line.trim();
  if (line.isEmpty()) return;
  int c = line.indexOf(':');
  String cmd = (c < 0 ? line : line.substring(0, c));
  String arg = (c < 0 ? "" : line.substring(c + 1));
  cmd.trim(); cmd.toLowerCase();
  bool authed = fromBle ? bleAuthed : serialAuthed;

  if (cmd == "auth") {
    bool ok = arg == setting("pin", DEVICE_PIN);
    if (fromBle) bleAuthed = ok;
    emitAck(cmd, ok, ok ? "unlocked" : "wrong PIN");
    return;
  }
  if (cmd == "status") { sendStatus(); return; }
  if (cmd == "ping")   { emitAck(cmd, true, "pong"); return; }

  // Everything below changes the device, so it needs the PIN over BLE.
  if (cmd == "set" || cmd == "wifi" || cmd == "reboot" || cmd == "get" || cmd == "forget") {
    if (!authed) { emitAck(cmd, false, "send auth:<PIN> first"); return; }
    if (cmd == "set") {
      int e = arg.indexOf('=');
      if (e <= 0) { emitAck(cmd, false, "use set:key=value"); return; }
      String k = arg.substring(0, e); k.trim();
      String v = arg.substring(e + 1); v.trim();
      if (k.length() > 15) { emitAck(cmd, false, "key too long"); return; }
      saveSetting(k.c_str(), v);
      emitAck(cmd, true, k + " saved");
      if (k == "ssid" || k == "pass") wifiConnect();
      // let the project react to changed thresholds etc.
      if (userHandler) userHandler("setting", k, true);
      return;
    }
    if (cmd == "get") {
      String v = setting(arg.c_str(), "");
      if (isSecretKey(arg) && v.length()) v = "****" + v.substring(max(0, (int)v.length() - 3));
      emitAck(cmd, true, arg + "=" + v);
      return;
    }
    if (cmd == "wifi") {
      int k = arg.indexOf(',');
      String ssid = k < 0 ? arg : arg.substring(0, k);
      String pass = k < 0 ? "" : arg.substring(k + 1);
      saveSetting("ssid", ssid); saveSetting("pass", pass);
      emitAck(cmd, true, "connecting to " + ssid);
      wifiConnect();
      return;
    }
    if (cmd == "forget") { prefs.clear(); emitAck(cmd, true, "all settings erased"); return; }
    if (cmd == "reboot") { emitAck(cmd, true, "rebooting"); delay(300); ESP.restart(); }
  }

  if (userHandler && userHandler(cmd, arg, authed)) return;
  emitAck(cmd, false, "unknown command");
}
```

```
// ------------------------------------------------------------ entry points
// Call once from setup().  id = short project id, e.g. "P01".
inline void begin(const char* id, CommandHandler handler) {
  Serial.begin(115200);
  delay(200);
  projectId = id;
  userHandler = handler;
  prefs.begin("iot", false);
  cmdQueue = xQueueCreate(8, sizeof(CmdMsg));
  WiFi.mode(WIFI_STA);
  WiFi.setAutoReconnect(true);
  // NOTE: Wi-Fi modem sleep must stay ON while BLE is running (ESP32 coexistence).
  bleBegin();
  wifiConnect();
  Serial.println("[CORE] ready - type 'status' here or connect over BLE");
}

// Call every loop() iteration.
inline void loop() {
  wifiLoop();
  // USB serial commands (handy while developing)
  static String serialLine;
  while (Serial.available()) {
    char ch = Serial.read();
    if (ch == '\n' || ch == '\r') { if (serialLine.length()) enqueue(serialLine, false); serialLine =
""; }
    else if (serialLine.length() < 250) serialLine += ch;
  }
  CmdMsg m;
  while (cmdQueue && xQueueReceive(cmdQueue, &m, 0) == pdTRUE) handleLine(String(m.text), m.fromBle);
}

// Round for compact JSON. NaN stays NaN, which ArduinoJson writes as null.
inline float rnd(float v, int dp = 1) {
  if (isnan(v)) return v;
  float m = powf(10, dp);
  return roundf(v * m) / m;
}

// Helper: build and send a {"type":"data",...} line from a filled document.
inline void sendData(JsonDocument& d) {
  d["type"] = "data";
  String s; serializeJson(d, s); emit(s);
}

inline void sendEvent(const String& name, const String& msg) {
  JsonDocument d;
  d["type"] = "event"; d["name"] = name; d["msg"] = msg; d["time"] = timestamp();
  String s; serializeJson(d, s); emit(s);
}

}  // namespace iot
```

Cloud.h

```
/*
 * Cloud.h  —  free-tier cloud helpers used by the projects
 * ---------------------------------------------------------
 *  ThingSpeak   : HTTPS REST "update" (free tier: 1 update / 15 s per channel)
 *  Adafruit IO  : MQTT over TLS (free tier: 30 data points / minute, 10 feeds)
 *  Supabase     : PostgREST over HTTPS (free tier: 500 MB Postgres)
 *
```

```
 *  Keys are read from settings (stored over BLE with set:key=value) so no
 *  secret has to be hard-coded or committed to GitHub:
 *     tskey    ThingSpeak Write API Key
 *     aiouser  Adafruit IO username        aiokey  Adafruit IO key
 *     sburl    https://<ref>.supabase.co   sbkey   Supabase publishable/anon key
 *
 *  TLS note: certificate checking is disabled with setInsecure() to keep the
 *  projects beginner-friendly and independent of CA rotations. Traffic is
 *  still encrypted. For production, pin the root CA with setCACert().
 */
#pragma once

#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <PubSubClient.h>
#include "IoTCore.h"

// Optional compile-time defaults (normally left empty and set over BLE)
#ifndef TS_WRITE_KEY
#define TS_WRITE_KEY ""
#endif
#ifndef AIO_USERNAME
#define AIO_USERNAME ""
#endif
#ifndef AIO_KEY
#define AIO_KEY ""
#endif
#ifndef SUPABASE_URL
#define SUPABASE_URL ""
#endif
#ifndef SUPABASE_KEY
#define SUPABASE_KEY ""
#endif

namespace cloud {

// ============================================================== ThingSpeak
// vals[i] -> field(i+1). Pass NAN to skip a field. Returns true on success.
inline bool thingSpeakUpdate(const float* vals, int n, const String& status = "") {
  if (!iot::wifiOK()) return false;
  String key = iot::setting("tskey", TS_WRITE_KEY);
  if (key.isEmpty()) { Serial.println("[TS] no tskey set"); return false; }
  String url = "https://api.thingspeak.com/update?api_key=" + key;
  for (int i = 0; i < n && i < 8; i++)
    if (!isnan(vals[i])) url += "&field" + String(i + 1) + "=" + String(vals[i], 2);
  if (status.length()) {
    String st = status; st.replace(" ", "%20");
    url += "&status=" + st;
  }
  WiFiClientSecure client; client.setInsecure();
  HTTPClient http; http.setTimeout(8000);
  if (!http.begin(client, url)) return false;
  int code = http.GET();
  String body = http.getString();
  http.end();
  bool ok = code == 200 && body.toInt() > 0;   // ThingSpeak returns the entry id, 0 = rejected
  Serial.printf("[TS] HTTP %d entry=%s\n", code, body.c_str());
  return ok;
}

// ============================================================== Supabase
// Generic PostgREST call. method: GET/POST/PATCH. path e.g. "/rest/v1/readings"
inline int supabaseRequest(const char* method, const String& path, const String& body,
```

```
                          String* response = nullptr, const char* prefer = "return=minimal") {
  if (!iot::wifiOK()) return -1;
  String base = iot::setting("sburl", SUPABASE_URL);
  String key  = iot::setting("sbkey", SUPABASE_KEY);
  if (base.isEmpty() || key.isEmpty()) { Serial.println("[SB] sburl/sbkey not set"); return -2; }
  if (base.endsWith("/")) base.remove(base.length() - 1);
  WiFiClientSecure client; client.setInsecure();
  HTTPClient http; http.setTimeout(8000);
  if (!http.begin(client, base + path)) return -3;
  http.addHeader("apikey", key);
  http.addHeader("Authorization", "Bearer " + key);
  http.addHeader("Content-Type", "application/json");
  if (prefer) http.addHeader("Prefer", prefer);
  int code = http.sendRequest(method, body);
  if (response) *response = http.getString();
  else if (code >= 300) Serial.printf("[SB] %s\n", http.getString().c_str());
  http.end();
  Serial.printf("[SB] %s %s -> %d\n", method, path.c_str(), code);
  return code;
}

inline bool supabaseInsert(const char* table, const String& json) {
  int c = supabaseRequest("POST", String("/rest/v1/") + table, json);
  return c >= 200 && c < 300;
}

// filter is a PostgREST filter, e.g. "id=eq.1"
inline bool supabaseUpdate(const char* table, const String& filter, const String& json) {
  int c = supabaseRequest("PATCH", String("/rest/v1/") + table + "?" + filter, json);
  return c >= 200 && c < 300;
}

// query e.g. "select=*&id=eq.1"  -> JSON array text in out
inline bool supabaseSelect(const char* table, const String& query, String& out) {
  int c = supabaseRequest("GET", String("/rest/v1/") + table + "?" + query, "", &out, nullptr);
  return c == 200;
}

// ============================================================ Adafruit IO
// MQTT wrapper with a simple rate limiter to respect the 30 msg/min limit.
using FeedCallback = void (*)(const String& feed, const String& value);

class AdafruitIO {
 public:
  // requestLast = true asks Adafruit IO for each feed's last value on connect
  // (useful for switches changed while offline; unsafe for things like pumps)
  void begin(FeedCallback cb, const char* const* subFeeds, int nSub, bool requestLast = true) {
    cb_ = cb; subs_ = subFeeds; nSub_ = nSub; requestLast_ = requestLast;
    tls_.setInsecure();
    mqtt_.setClient(tls_);
    mqtt_.setServer("io.adafruit.com", 8883);
    mqtt_.setBufferSize(512);
    mqtt_.setKeepAlive(60);
    mqtt_.setCallback([](char* topic, byte* payload, unsigned int len) {
      String t(topic), v;
      for (unsigned int i = 0; i < len; i++) v += (char)payload[i];
      int p = t.lastIndexOf('/');
      String feed = t.substring(p + 1);
      if (instance()->cb_) instance()->cb_(feed, v);
    });
    instance() = this;
  }
```

```
  void loop() {
    if (!iot::wifiOK()) return;
    if (!mqtt_.connected()) {
      if (millis() - lastTry_ < 10000) return;
      lastTry_ = millis();
      user_ = iot::setting("aiouser", AIO_USERNAME);
      String key = iot::setting("aiokey", AIO_KEY);
      if (user_.isEmpty() || key.isEmpty()) { Serial.println("[AIO] aiouser/aiokey not set"); return; }
      String id = "esp32-" + String((uint32_t)ESP.getEfuseMac(), HEX);
      Serial.println("[AIO] connecting...");
      if (mqtt_.connect(id.c_str(), user_.c_str(), key.c_str())) {
        Serial.println("[AIO] connected");
        for (int i = 0; i < nSub_; i++) {
          mqtt_.subscribe(topic(subs_[i]).c_str());
          if (requestLast_) mqtt_.publish((topic(subs_[i]) + "/get").c_str(), "");
        }
        // subscribe to throttle/error topics so we learn about rate limits
        mqtt_.subscribe((user_ + "/throttle").c_str());
      } else {
        Serial.printf("[AIO] failed, state %d\n", mqtt_.state());
      }
    }
    mqtt_.loop();
  }

  bool connected() { return mqtt_.connected(); }

  // Returns false if not connected or if the per-minute budget is used up.
  bool publish(const char* feed, const String& value) {
    if (!mqtt_.connected()) return false;
    uint32_t now = millis();
    if (now - windowStart_ > 60000) { windowStart_ = now; sent_ = 0; }
    if (sent_ >= 28) { Serial.println("[AIO] rate budget used, skipping"); return false; }
    sent_++;
    return mqtt_.publish(topic(feed).c_str(), value.c_str());
  }
  bool publish(const char* feed, float v, int dp = 2) { return publish(feed, String(v, dp)); }

 private:
  String topic(const char* feed) { return user_ + "/feeds/" + feed; }
  static AdafruitIO*& instance() { static AdafruitIO* p = nullptr; return p; }

  WiFiClientSecure tls_;
  PubSubClient mqtt_;
  FeedCallback cb_ = nullptr;
  const char* const* subs_ = nullptr;
  int nSub_ = 0;
  bool requestLast_ = true;
  String user_;
  uint32_t lastTry_ = 0, windowStart_ = 0;
  int sent_ = 0;
};

}  // namespace cloud
```